Better Name for Better Lookups in P3091

Document #:	P4139R2 [Latest] [Status]
Date:	2026-05-08 22:25 EDT
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Nathan Myers <ncm@cantrip.org> Pablo Halpern <phalpern@halpernwightsoftware.com>

§ 1 Abstract

[P3091R5], forwarded to LWG for C++29, adds member functions `get` to associative containers in the Standard Library. These member functions work like `operator[]` or `at`, but return an `optional<T&>` value instead of a simple reference; when lookup fails, they return the empty object. While the feature was strongly favored, support for the name `get` yielded only a weak consensus, with opposition to the name focusing on the fact that elsewhere in the Standard Library, `get` calls (with one exception) cannot fail or take not-strictly-bounded time.

In hope of making the C++26 cutoff, the naming discussion was curtailed and the name in P3091 was retained when that paper was forwarded in Sofia. Since that time, even the author of P3091 has reconsidered the implications of the name `get`. As LWG has not yet taken up the paper (which is now on track for C++29) we present an analysis of the status quo and propose alternative names for the desired functionality.

§ 2 Change Log

— R2 - Pre-Brno Mailing, 2026-05: Add survey of existing usage of `get`, comparisons among proposed alternatives, and wording.

— R1 - Post-Croydon Mailing, 2026-04: Add Pablo Halpern as co-author; identify correct (C++29) target; add to other indexable containers; suggest when operations should be no-throw.

— R0 - Croydon 2026-03, posted during the meeting.

§ 3 Motivation

§ 3.1 Current Usage of `get` in the Standard Library

In the current Working Draft, the Standard Library defines functions `get` in `<array>`, `<complex>`, `<format>`, `<functional>`, `<future>`, `<memory>`, `<ranges>`, `<tuple>`, `<utility>`, and `<variant>` (neglecting I/O and locale, where `get` just complements `put`) and, notably, *none* of the growable containers. Every `get` function in these headers has the following properties:

— It returns a reference to or copy of a subobject, performing a monadic unwrapping.

— Its returned value is already present, admitting no failure. Even when `shared_ptr<>::get` or `unique_ptr<>::get` return `nullptr`, that is the actual value wrapped.

— When it returns without throwing, the operation was successful. In most cases, it cannot fail.

— It does essentially no computation to obtain the value. Most know the location at compile time; the least disciplined, `format_args::get(i)`, does an indexed array lookup. `future::get` may block or throw, but if the computation completed successfully, it returns immediately. (Pablo wishes it known that he considers this last property of little interest.)

The library has accumulated an assortment of `try_`*`xxxxx`* members that share the feature of returning in the case of failure, but yield a variety of result types, some upon failure presenting an iterator adjacent to where an element *would have* been inserted or erased, or `end()`, or a default-constructed iterator in a pair with a boolean flag. They generally cost less on failure than the non-`try` alternative, where there is one. Now that returning an `optional<T&>` is possible, it is favored for these.

For decades before those, associative containers have had `at(key)` that throws on a lookup failure, and `operator[](key)` that inserts a default-initialized value and returns a reference to it, but also might throw if that insertion fails. That these return an element value reference makes them close kin to the feature adopted from [P3091R5].

§ 3.2 Conclusions Drawn from Current Usage

We conclude that a `get` function returning `optional` after an unbounded search would make it unique among uses of `get` in the library, and so would be inconsistent and confusing. (Pablo again emphasizes that he does not consider search time relevant here.)

The name of the desired operation should reflect that the lookup may take variable time and could return without finding the value being sought; existing instances of `get` in the Library forbid both.

P3091 offered names `get_optional` and `lookup_optional`, which failed to evoke enthusiasm in Sofia; and `lookup`, which had supporters, but failed to reach consensus. Interest at the time was focused on semantics, with discussion of names felt by many to be a distraction. The only criterion mentioned for a good name was that it be short, which seems to the authors secondary to descriptive clarity and consistency.

The goal of this paper is to propose a short list of names which meet the criteria of being short, accurately descriptive, and consistent (or at least not inconsistent) with the names of other value-or-reference-returning functions that report failure via a non-throwing path.

§ 4 Proposed Change

This is a multipart proposal. The parts can be polled separately.

§ 4.1 A Quick Review of P3091

[P3091R5] proposed a new member function for associative containers returning an `optional<mapped_type&>` that identifies an object if the key is found, or otherwise is empty. A typical use might look like:

```
if (auto maybe = container.xxxxx(key)) {
  use(*maybe);
}
```

or

```
use(container.xxxxx(key).value_or(mapped{}));
```

§ 4.2 Part 1: Change *map-like*`::get` to a different name in P3091

It is common to downplay the importance of naming, but bad names do real harm. Naming discussions should not be an exercise in throwing a bunch of names in the air and voting based on gut reactions. We propose that LEWG choose among the three alternatives below. When polling the alternatives, we should pay attention not only to the name that receives the most positive votes, but the opposition to any specific name.

Name	Discussion
`try_at`	Implies `at`-like functionality, replacing throwing with `optional<T&>` as the failure channel.
`lookup`	Simple and does not imply success. (Pablo's preference)
`try_lookup`	The `try_` prefix hints that if lookup fails it does not throw, consistent with other instances such as `inplace_vector<T>::try_push`.

Recall that the operation does not throw unless a user-supplied function object it calls (comparison, hash, etc) throws.

This change is targeted for C++29. Note that P3091 has not yet been voted into the working paper. This proposal would not change anything in the C++26 DIS.

§ 4.3 Part 2: Add a Similar Member Function to Random-access Sequence Containers

Conceptually, `myVector[index]` is a lookup operation that could fail if *`index`* is out of range, so member functions similar to the those added to associative containers would fit well in sequence containers, adding to the regularity of the library.

We therefore propose that such member functions, using the name chosen, be added to random-access containers (and to any containers exposing an `operator[](index)` or `at(index)` member).

§ 5 Alternatives considered

The name `lookup_optional` meets the stated criteria, except for its length. If there is sufficient interest, it might be considered as well. The name `get_optional` mentioned in P3091 is not proposed, for reasons that should be clear.

§ 6 Wording

§ 6.1 Part 1

Generate a new revision of P3091, replacing `get` with the name selected, including in the feature-test macro.

If P3091 is voted into the WD before this paper is forwarded to LWG, the change would apply to the `get` functions in the associative containers in the WD, instead.

§ 6.2 Part 2

Wording for Part 2 of this proposal is relative to the 2025-12 Working Draft, [N5032].

Let *newname* be `try_at`, `lookup`, or `try_lookup`, or as determined in part 1, above.

At the end of 23.2.4 [sequence.reqmts][1], add:

> `a.` *newname* `(n)`
>
> *Result* (`R`): `optional<mapped_type&>`; `optional<const mapped_type&>` for constant `a`.
>
> *Returns*: `n < a.size() ? R{a[n]} : R{}`
>
> *Throws*: nothing
>
> *Remarks*: Required for `basic_string`, `array`, `deque`, `inplace_vector`, and `vector`.

In 23.3.5.1 [deque.overview], add the _newname` members:

```
  // element access
  constexpr reference operator[](size_type n);
  constexpr const_reference operator[](size_type n) const;
  constexpr reference at(size_type n);
  constexpr const_reference at(size_type n) const;
  constexpr optional<mapped_type&> newname(size_type n) noex-
cept;
  constexpr optional<const mapped_type&> newname(size_type n)
const noexcept;
  constexpr reference front();
  constexpr const_reference front() const;
constexpr reference back();
constexpr const_reference back() const;
```

Make corresponding changes in 27.4.3.1 [basic.string.general], 23.3.3.1 [array.overview], 23.3.16.1 [inplace.vector.overview], and 23.3.13.1 [vector.overview].

§ 7 References

[N5032] Thomas Köppe. 2025-12-15. Working Draft, Standard for Programming Language C++.
https://wg21.link/n5032

[P3091R5] Pablo Halpern. 2025-10-03. Better lookups for `map`, `unordered_map`, and `flat_map`.
https://wg21.link/p3091r5

---

1. All citations to the Standard are to working draft N5032 unless otherwise specified.↩